



Hochschule für Technik,
Wirtschaft und Kultur Leipzig

FAKULTÄT INGENIEURWISSENSCHAFTEN

AUTOMATISIERUNGS - MASTER

Embeddedsystems 2

Autoren B.Eng Lukas Blechschmidt

B.Eng Nico Kirsch

Betreuer Prof. Dr.-Ing. Andreas Pretschner

2026-04-30

Inhaltsverzeichnis

1	Aufgabenstellung	3
1.1	Umsetzung der Programmieraufgabe	4
1.2	Umsetzung des Belegs	4
2	Umsetzung des Watchdogs	5
2.1	heartbeat.cpp	6
2.2	main.cpp	7
2.3	mock-hal.cpp	9
3	Autotools	10
3.1	Projektstruktur und Organisation	10
3.1.1	Makefile.am	10
3.1.2	src/Makefile.am	10
3.1.3	configure.ac	11
3.2	Ausführen von Autotools	11
3.3	Ergebnis	12
4	Manuelles Makefile	13
4.1	Makefile	13
4.2	Vergleich zu Autotools	15
5	STM32 Portierung und Cross-Compilation	16
5.1	Anpassungen am Quellcode	16
5.2	Anpassungen am manuellen Makefile	16
5.3	Anpassungen in Autotools	17
5.3.1	config.ac	18
5.3.2	Makefile.am	18
6	Auswertung	20

Abbildungsverzeichnis

Abbildung 1 Aufgabenstellung - spezifische Anforderungen	3
Abbildung 2 Heartbeat - Output	12

1 Aufgabenstellung

Es soll ein Heartbeat + *Watchdog* auf Basis eines STM32 Boards implementiert werden. Um die Ausführung eines Hauptprogrammes zu simulieren, soll eine LED blinken. Während dieses Prozesses soll ein *Watchdog* die korrekte Ausführung dieses Hauptprogrammes überprüfen. Sollte ein Fehler in dem Programm auftreten, erkennt der *Watchdog* dies durch Ausbleiben der Bestätigung und der Mikrocontroller wird neugestartet. Bei simplen Programmen, wie dem Blinken einer LED ist dies nicht zwangsläufig nötig. Bei komplexeren Programmen, mit Verwendung von If-Else Abfragen oder Schleifen ohne Unterbrechung könnte dies sinnvoll sein, damit der Mikrocontroller aus Fehlerfällen eigenständig herauskommt. Dies verhindert nicht das Eintreten des Fehlers selbst, aber kann zur Behebung beitragen, vor allem wenn der auslösende Fehler nur selten auftritt. Das Projekt umfasst dabei folgende spezifische Anforderungen:

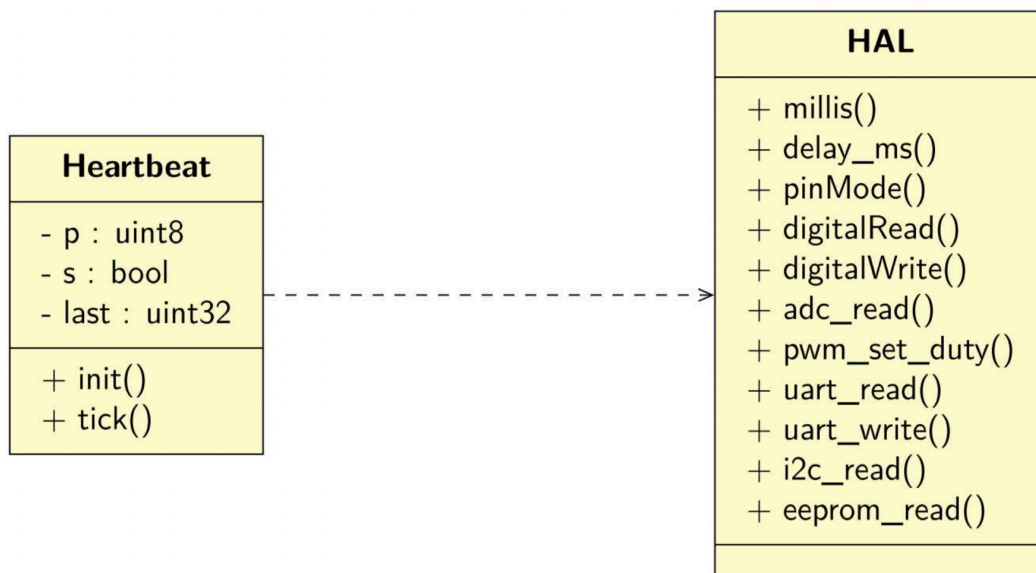


Abbildung 1: Aufgabenstellung - spezifische Anforderungen

- **Inputvariablen:** Implementierung des *Watchdog*-Intervalls, das als zentraler Parameter für die Überwachung dient.
- **Funktionalität:** Das Programm soll eine LED blinken lassen und den Controller neustarten, falls der *Watchdog* auslöst. Dies dient der visuellen Rückmeldung und der Systemstabilität.
- **HAL-Bibliothek:** Nutzung einer HAL-Bibliothek des STM32, wobei HAL-spezifische Funktionen wie `HAL_Delay` und `HAL_GPIO_WritePin` verwendet werden. Diese Funktionen ermöglichen eine effiziente und hardwarenahe Steuerung. Die Funktionen wurden durch Dummyfunktionen ausgetauscht, um eine Ausgabe in der Konsole zu realisieren.

1.1 Umsetzung der Programmieraufgabe

Für die Zusammenarbeit wurde der Datenaustausch über *GitHub* realisiert. Dazu wurde in *GitHub* ein Repository angelegt, in welchem sich immer der aktuelle Stand der Ausarbeitung befindet. Dieses Repository kann über den folgenden Link erreicht werden:

<https://github.com/NicoKirsch/Embedded-2.git>

Dadurch ist eine korrekte Versionierung der Ausarbeitung sichergestellt.

1.2 Umsetzung des Belegs

Der Beleg wurde nicht über *GitHub* erstellt. Dafür wurde ein cloudbasierter TeX-Editor verwendet. In diesem konnte dann die Erarbeitung ähnlich simultan erfolgen. Ebenfalls wurde hier wie bei *GitHub* verhindert, dass mit veralteten Versionen gearbeitet wird.

Der Editor hat auch das Kompilieren übernommen, welches direkt mit dem Schreiben des Texts erfolgte. Dadurch gab es immer eine optische Ausgabe des gerade eingebauten Abschnittes. Dies ist ein entscheidender Vorteil gegenüber installierter Software wie *TeXStudio*, bei der die einzelnen Dateien vorher immer händisch synchronisiert und ein Compilevorgang manuell ausgelöst werden muss.

Die sprachliche Qualitätssicherung und Strukturierung dieses Berichts erfolgte mit Unterstützung von Künstlicher Intelligenz. Sämtliche technischen Inhalte und die Implementierung des *Watchdogs* liegen in der Verantwortung der Autoren.

2 Umsetzung des Watchdogs

Für das Lösen der Aufgabenstellung wurde zunächst die folgende Ordnerstruktur angelegt:

```
1 autotools-project/  
2 |— configure.ac  
3 |— Makefile.am  
4 |— include/  
5 |   |— mock_hal.hpp  
6 |   |— heartbeat.hpp  
7 |— src/  
8 |   |— main.cpp  
9 |   |— mock_hal.cpp  
10  |— heartbeat.cpp
```

Da für die Implementierung eines manuellen Makefiles die Dateien **Makefile.am** und **configure.ac** nicht benötigt werden, wurden diese entfernt und durch ein einfaches **Makefile** im Hauptverzeichnis ersetzt.

Da die STM32 Bibliothek, die für die Programmierung des Nucleo432 notwendig wäre einen zu großen Umfang besitzt, wurde die **mock_hal.cpp** implementiert. Diese **mock_hal** beinhaltet nur die im Projekt genutzten HAL Befehle, um die Komplexität zu verringern. Die Befehle in der mock_hal erzeugen Konsolenausgaben anstelle der Ansteuerung von Werten im Mikrocontroller. Somit wird es möglich, das Programm auch ohne Hardware funktional zu prüfen. Das Programm kann auf diese Weise jedoch zunächst nur auf Unix Systemen genutzt werden. Für die spätere Verwendung des Programms auf einem STM32 sind Anpassungen in der Bibliothek notwendig oder die Bibliothek muss durch die originale STM32 HAL-Bibliothek ausgetauscht werden.

2.1 heartbeat.cpp

```
1 #include "heartbeat.hpp"
2 #include "mock_hal.hpp"
3
4 void Heartbeat::init(uint32_t t_ms) {
5     p = t_ms; // initialisieren des Watchdog (nach wievielen Sekunden
        wird der Watchdog getriggert)
6     s = false; // Watchdog nicht ausgelöst
7     last = HAL_GetTick(); // aktueller Zeitpunkt in ms
8 }
9
10 void Heartbeat::tick() {
11     // Überprüfen, ob Watchdog innerhalb der Zeit getriggert
12     auto now = HAL_GetTick(); // aktueller Zeitpunkt in ms
13     if (now - last >= p) { // wenn mehr als p ms vergangen ist,
        Watchdog auslösen
14         // stm32 Watchdog auslösen
15         s = true; // Watchdog ausgelöst
16         NVIC_SystemReset();
17     }
18     last = now; //aktueller Zeitpunkt in ms
19 }
```

Der Watchdog wurde in der Datei *heartbeat.cpp* implementiert. Über die *init*-Funktion lässt sich das Zeitintervall des Watchdogs vorgeben, welches die maximal zulässige Zeitspanne zwischen zwei *tick()*-Aufrufen definiert.

Die *tick()*-Funktion prüft, ob seit ihrem letzten Aufruf mehr Zeit vergangen ist, als die zuvor definierte Periode *p*. Ist dies der Fall, wird die Funktion **NVIC_SystemReset()** aufgerufen, was im Kontext des STM32 Nucleo-Boards einen Neustart des Mikrocontrollers auslösen würde. Somit wird sichergestellt, dass das System bei einer zu langen Programmlaufzeit, was auf einen Systemfehler oder Blockade hindeutet, automatisch zurückgesetzt wird. Im Programm wird zunächst nur eine Konsolenausgabe im Fall des Zurücksetzens erzeugt.

2.2 main.cpp

```
1 #include "heartbeat.hpp"
2 #include "mock_hal.hpp"
3
4 int main() {
5
6     Heartbeat heart;
7
8     // Watchdog initialisieren (Beispiel: 200ms)
9     heart.init(200);
10
11     int loopCounter = 0; // Zähler für Anzahl der Schleifendurchläufe
12
13     while (true) {
14         HAL_GPIO_WritePin(LEDPIN_GPIO_Port, LEDPIN_Pin, GPIO_PIN_SET);
15         HAL_Delay(1);
16
17         // Watchdog triggern
18         heart.tick();
19
20         HAL_GPIO_WritePin(LEDPIN_GPIO_Port, LEDPIN_Pin, GPIO_PIN_RESET);
21         HAL_Delay(1);
22
23         // Watchdog triggern
24         heart.tick();
25
26         loopCounter++;
27         if (loopCounter == 10) { // nach 10 Schleifendurchläufen den
Watchdog nicht mehr triggern
28             HAL_Delay(2000); // Verzögerung auslösen; Restart
triggern
29         }
30     }
31     return 0;
32 }
```

Die *main*-Funktion lässt eine LED kontinuierlich blinken, um den Normalbetrieb des Systems optisch zu signalisieren. Auch hier wird das Blinken im Terminal realisiert. Gleichzeitig ruft sie regelmäßig die *tick()*-Funktion des Watchdogs auf, um einen Neustart zu verhindern und die korrekte, fortlaufende Ausführung des Programms zu bestätigen. In dem Code blinkt die LED mit einer Frequenz von 500 Hz, also besitzt sie eine Ein-Zeit und Aus-Zeit von 1 ms. Nach jedem Schaltvorgang wird ein Signal an den Watchdog gesendet, der die letzte Ausführzeit prüft und die aktuelle speichert. Der Watchdog selber löst erst nach 200 ms keiner Rückmeldung aus, was das 200-fache eines korrekten Signals darstellt. Diese Zeiten sind in dem Beispiel absichtlich extrem gewählt und sollten in einer reellen Anwendung zum Anwendungsfall entsprechend gewählt werden. Zum Beispiel könnte der Watchdog bereits bei 3- oder 5-facher Verlangsamung auslösen. Dann könnte schon auf kleinere Fehler reagiert werden, allerdings würden dann rechenintensive Aufgaben mit variabler Zeit eventuell auch einen Neustart triggern. Komplexe

Rechnungen müssen somit Watchdog-Rückmeldungen integrieren. In dem Anwendungsfall könnte man auch nur einen Trigger nach jedem Einschalten an den Watchdog senden, was die Rechenlast durch diese eine Aufgabe reduziert, was in Embedded Systemen relevant werden kann.

Um einen Fehlerfall und das Eingreifen des Watchdogs zu simulieren, wurde ein Zähler (loopCounter) in Kombination mit einer if-Bedingung implementiert. Nach zehn regulären Durchläufen der Hauptschleife wird künstlich eine Verzögerung von 2000 ms erzeugt. Dadurch erfolgt die Rückmeldung an den Watchdog zu spät. Das Zeitintervall überschreitet die erlaubten 200 ms, wodurch das System wie gewünscht zurückgesetzt wird.

2.3 mock-hal.cpp

```
1 #include "mock_hal.hpp"
2 #include <iostream>
3 #include <chrono>
4 #include <thread>
5
6 uint32_t HAL_GetTick() {
7     // gibt Zeit seit Start in Millisekunden zurück
8     auto now = std::chrono::steady_clock::now().time_since_epoch();
9     return
10     std::chrono::duration_cast<std::chrono::milliseconds>(now).count();
11 }
12
13 void HAL_Delay(uint32_t ms) {
14     std::this_thread::sleep_for(std::chrono::milliseconds(ms));
15 }
16
17 void HAL_GPIO_WritePin(const char* port, int pin, int state) {
18     std::cout << "[HAL] Port: " << port << " Pin: " << pin
19     << " State: " << (state ? "HIGH" : "LOW") << std::endl;
20 }
21
22 void NVIC_SystemReset() {
23     std::cout << "[HAL] System Reset triggered!" << std::endl;
24 }
```

Zur Nachbildung der HAL wurde die Datei `mock_hal.cpp` erstellt. Diese stellt im Wesentlichen Dummy-Funktionen der für den STM32 benötigten HAL-Bibliothek bereit. Anstatt direkt auf die Hardware zuzugreifen, greift diese deutlich vereinfachte Version beispielsweise auf die Standardbibliothek für Zeit- und Konsolenausgaben zurück. Diese Abstraktion ermöglicht es, die Kernlogik des Watchdogs zunächst komfortabel auf dem Entwicklungsrechner in der Konsole zu testen, ohne vorab die komplexe Integration der echten STM32-HAL-Bibliothek auf der Zielhardware vornehmen zu müssen.

3 Autotools

Das Projekt wurde zunächst mit den *GNU Autotools* realisiert. Dieses Build-System automatisiert die Erstellung von Makefiles und erleichtert die Portabilität des Quellcodes auf verschiedene Systeme.

3.1 Projektstruktur und Organisation

3.1.1 Makefile.am

```
1 SUBDIRS = src
```

Diese Datei befindet sich im Hauptverzeichnis. Mit der Zuweisung *SUBDIRS = src* wird Automake angewiesen, beim Build-Prozess auch das Unterverzeichnis *src* rekursiv zu durchlaufen und das dortige Makefile zu verarbeiten.

3.1.2 src/Makefile.am

```
1 bin_PROGRAMS = heartbeat_sensor
2 heartbeat_sensor_SOURCES = main.cpp heartbeat.cpp mock_hal.cpp
3 heartbeat_sensor_CPPFLAGS = -I$(top_srcdir)/include
4 heartbeat_sensor_CFLAGS = -I$(top_srcdir)/include
```

In dieser Datei wird genau definiert, wie das Programm kompiliert werden soll:

- **bin_PROGRAMS:** Hiermit wird der Namen der ausführbaren Datei definiert, die am Ende generiert werden soll.
- **heartbeat_sensor_SOURCES:** Diese Liste beinhaltet alle Quellcodedateien auf, die für die Erstellung des Programms kompiliert und gelinkt werden müssen.
- **heartbeat_sensor_CPPFLAGS:** Dadurch findet der Compiler die .hpp-Headerdateien.
- **heartbeat_sensor_CFLAGS:** Die CFLAGS verhalten sich äquivalent zu den CPPFlags, jedoch für den C-Compiler. Da die STM32 Bibliothek später in C definiert ist, ist diese Angabe hilfreich.

3.1.3 configure.ac

```
1 AC_INIT([heartbeat_sensor], [1.0], [you@example.com])
2 AM_INIT_AUTOMAKE([-Wall -Werror foreign])
3 AC_PROG_CC
4 AC_PROG_CXX
5 AC_CONFIG_SRCDIR([src/main.cpp])
6 AC_CONFIG_HEADERS([config.h])
7 AC_CONFIG_FILES([Makefile
8 src/Makefile])
9 AC_OUTPUT
```

Die *configure.ac* ist das Herzstück von Autoconf und prüft das System auf vorhandene Compiler und Bibliotheken.

- **AC_INIT:** Initialisiert Autoconf mit dem Projektnamen (*heartbeat_sensor*)
- **AM_INIT_AUTOMAKE:** Dieser Befehl initialisiert Automake und die anschließenden Parameter schalten alle Warnungen ein (*-Wall*).
- **AC_PROG_CC / AC_PROG_CXX:** Sie prüft, ob ein funktionierender C- und C++-Compiler auf dem System vorhanden ist und konfiguriert diesen für den Build.
- **AC_CONFIG_SRCDIR:** Dieser Sicherheits-Check überprüft, ob der angegebene Pfad (*src/main.cpp*) tatsächlich existiert. Damit wird sichergestellt, dass das Skript im richtigen Verzeichnis ausgeführt wird.
- **AC_CONFIG_HEADERS:** Es generiert eine *config.h*-Headerdatei, in der plattformspezifische Definitionen und Konfigurationen abgelegt werden.
- **AC_CONFIG_FILES:** Definiert, welche Makefiles generiert werden sollen.
- **AC_OUTPUT:** Hiermit erfolgt der Abschluss des Konfigurationsskriptes und es wird die tatsächliche Generierung der festgelegten Dateien ausgelöst.

3.2 Ausführen von Autotools

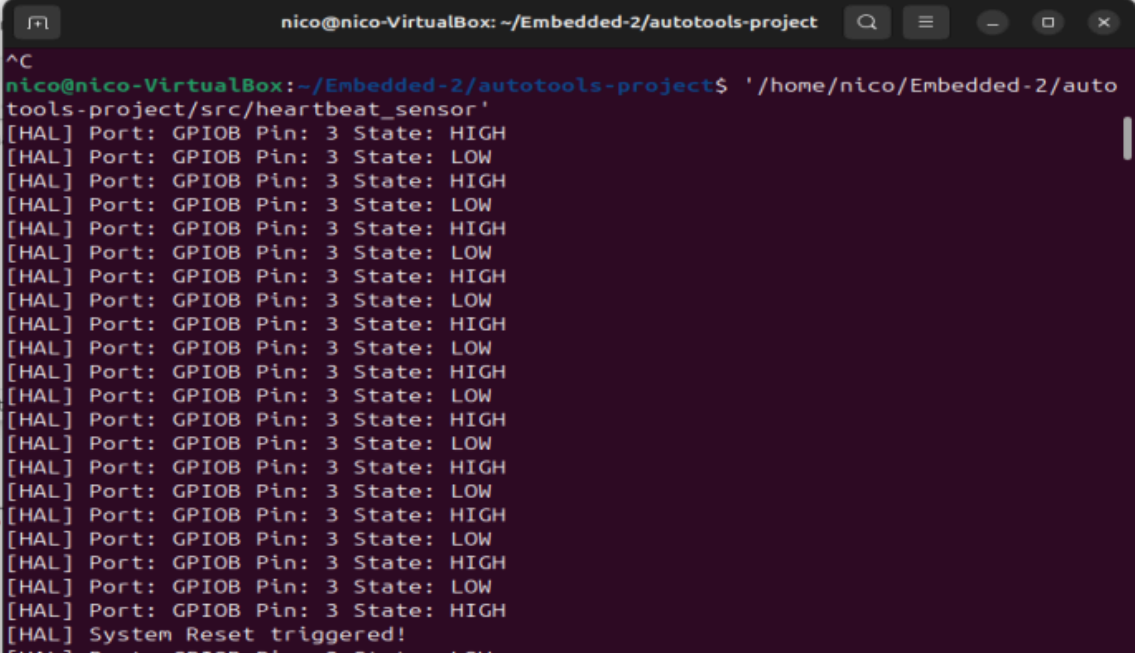
```
1 autoreconf -i
2 ./configure
3 make
```

Um das Projekt zu bauen, werden diese drei Befehle nacheinander ausgeführt:

- **autoreconf -i:** Liest die Dateien *configure.ac* sowie *Makefile.am* ein und generiert daraus das *configure*-Skript sowie die nötigen Template-Dateien (*Makefile.in*).
- **./configure:** Analysiert das lokale System (Compiler, Bibliotheken) und erzeugt anhand der Templates die finalen Makefiles, die exakt auf die aktuelle Umgebung zugeschnitten sind.
- **make:** Startet den eigentlichen Kompilierungsvorgang anhand der zuvor generierten Makefiles und erzeugt die ausführbare Datei *heartbeat_sensor*.

3.3 Ergebnis

Wird die kompilierte ausführbare Datei in der Konsole gestartet, wird das Ein- und Ausschalten der LED in der Konsole simuliert und entsprechend textuell ausgegeben. Nach genau 10 Schleifendurchläufen greift der absichtlich eingefügte Fehler: Der Watchdog wird nicht mehr rechtzeitig getriggert, woraufhin das System einen theoretischen Systemreset textuell auf der Konsole meldet.



```
nico@nico-VirtualBox: ~/Embedded-2/autotools-project
^C
nico@nico-VirtualBox:~/Embedded-2/autotools-project$ './home/nico/Embedded-2/autotools-project/src/heartbeat_sensor'
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] Port: GPIOB Pin: 3 State: LOW
[HAL] Port: GPIOB Pin: 3 State: HIGH
[HAL] System Reset triggered!
```

Abbildung 2: Heartbeat - Output

Somit ist die grundlegende Logik und Programmfunktion nachgewiesen. Die Autotools-Konfiguration arbeitet fehlerfrei und das Projekt kann im nächsten Schritt für die tatsächliche Hardware, den STM32-Mikrocontroller, angepasst werden.

4 Manuelles Makefile

Neben der Implementierung von einer Autotools Toolchain wurde außerdem ein manuelles Makefile erzeugt.

4.1 Makefile

```
1 CXX = g++
2 CXXFLAGS = -I./include -Wall -Wextra -std=c++11
3
4 SOURCES = src/main.cpp src/heartbeat.cpp src/mock_hal.cpp
5 OBJECTS = $(SOURCES:.cpp=.o)
6 TARGET = heartbeat_sensor
7
8 all: $(TARGET)
9
10 $(TARGET): $(OBJECTS)
11     $(CXX) $(CXXFLAGS) -o $@ $^
12
13 %.o: %.cpp
14     $(CXX) $(CXXFLAGS) -c $< -o $@
15
16 clean:
17     rm -f $(OBJECTS) $(TARGET)
18
19 .PHONY: all clean
```

Das Makefile gliedert sich in Variablen zur Konfiguration sowie Regeln für den Build-Vorgang:

Variablen-Definitionen:

- **CXX = g++:** Legt den zu verwendenden C++-Compiler fest.
- **CXXFLAGS:** Definiert die Compiler-Optionen. `-I./include` fügt das Header-Verzeichnis zum Suchpfad hinzu, `-Wall -Wextra` aktiviert zusätzliche Warnmeldungen zur Code-Qualität, und `-std=c++11` erzwingt den C++11-Standard.
- **SOURCES & OBJECTS:** **SOURCES** listet alle Quelldateien auf. **OBJECTS** konvertiert diese Liste automatisch, indem die Endung `.cpp` durch `.o` (Objektdateien) ersetzt wird.
- **TARGET:** Definiert den Namen der endgültigen ausführbaren Datei (`heartbeat_sensor`).

Build-Regeln:

- **all: \$(TARGET):** Dies ist das Standard-Ziel. Wenn man einfach `make` aufruft, wird dieses Ziel angesprochen, welches wiederum vom `TARGET` abhängt.
- **\$(TARGET): \$(OBJECTS):** Diese Regel beschreibt das Linken. Sie nimmt alle erzeugten Objektdateien und bindet sie mit dem Compiler `$(CXX)` zur fertigen Programmdatei `$(@)` zusammen.
- **%.o: %.cpp:** Eine Musterregel, die definiert, wie jede einzelne `.cpp`-Datei in eine `.o`-Datei kompiliert wird (`-c` bedeutet hierbei „compile only, do not link“).
- **clean:** Dieses Ziel dient dem Aufräumen. Es löscht mit `rm -f` alle erstellten Objektdateien und das Programm, um einen sauberen Zustand für den nächsten Build zu gewährleisten.

Spezielle Direktive:

- **.PHONY:** `all clean`: Dies weist `make` an, dass `all` und `clean` keine echten Dateien auf der Festplatte repräsentieren, sondern reine Befehls-Ziele sind. Dies verhindert Konflikte, falls es einmal eine Datei namens „clean“ im Ordner geben sollte.

Das Makefile wird durch den Konsolenbefehl **make** ausgeführt. Dabei wird automatisch das Ziel `all` angesteuert, welches die Quelldateien kompiliert und eine auf Linux ausführbare Datei erstellt.

4.2 Vergleich zu Autotools

Die Unterschiede zwischen den beiden Vorgehensweisen äußern sich primär nicht in der unmittelbaren Funktionsfähigkeit des Programms, sondern in der Robustheit und Flexibilität des Build-Systems.

Der offensichtlichste Unterschied der beiden Vorgehensweise ist die Größe des jeweiligen Makefiles. Während das manuell erstellte Makefile lediglich 19 Zeile Code benötigt, erzeugt Autotools ein Makefile mit 776 Zeilen.

Das manuell erstellte Makefile bietet eine direkte Kontrolle über den Prozess und bleibt aufgrund seiner geringen Komplexität sehr übersichtlich. Sobald jedoch komplexere Abhängigkeiten oder unterschiedliche Zielplattformen hinzukommen, stößt dieser manuelle Ansatz an seine Grenzen. Makefiles erfordern für jedes System händisch angepasste Pfade, Compiler-Flags und Bibliotheks-Suchpfade. Der Hauptvorteil der GNU Autotools liegt also in der automatisierten Portierbarkeit. Während das manuelle Makefile fest „hardcodierte“ Annahmen über das System trifft, führt das durch Autotools generierte ./configure-Skript beim Aufruf eine detaillierte Systemanalyse durch:

- **System-Infrastruktur:** Es prüft, ob benötigte Compiler, Bibliotheken und Header-Dateien vorhanden sind und wo sich diese im Dateisystem befinden.
- **Plattform-Unabhängigkeit:** Durch das .configure-Skript wird das Build-System exakt an das Zielsystem angepasst, ohne dass der Quellcode oder die Build-Definitionen manuell modifiziert werden müssen.
- **Fehlersicherheit:** Während das manuelle Makefile sich auf die grundlegenden Schritte des Kompilierens beschränkt, erzeugt Autotools eine Vielzahl an automatisierten Tests und Fallback-Mechanismen. Dies gewährleistet einen stabilen Compile-Prozess, selbst wenn das Zielsystem in Details von der Entwicklungsumgebung abweicht.

Zusammenfassend lässt sich sagen: Das manuelle Makefile ist für den schnellen Einstieg und lokale Tests hervorragend geeignet. Die Autotools hingegen sind das Werkzeug der Wahl für Softwareprojekte, bei denen eine hohe Kompatibilität über verschiedene Betriebssysteme hinweg erforderlich ist. Sie reduzieren den Wartungsaufwand bei Portierungen massiv, auch wenn die initiale Konfiguration durch configure.ac und Makefile.am eine höhere Komplexität aufweist.

5 STM32 Portierung und Cross-Compilation

Um den zuvor in der Konsole validierten Code auf einem STM32-Mikrocontroller auszuführen, sind grundlegende Anpassungen an der Architektur und der Build-Umgebung notwendig.

5.1 Anpassungen am Quellcode

Da die hardwarenahen Funktionen der *mock_hal.cpp* (wie *std::cout* oder *std::thread*) auf einem Mikrocontroller ohne Betriebssystem nicht existieren, wurden diese Befehle in der *mock_hal.cpp* entfernt. An ihre Stelle müssten in einer produktiven Umgebung spezifische STM32-HAL-Aufrufe eingesetzt werden. Da die vollständige Einbindung der STM32-Bibliothek den Rahmen dieser Aufgabenstellung überstiegen hätte, verbleiben die Funktionen in der *mock_hal.cpp* funktionslos. Dies ermöglicht es jedoch, das grundlegende Konzept des Watchdogs und das Prinzip des Cross-Compilings erfolgreich zu demonstrieren.

5.2 Anpassungen am manuellen Makefile

Da die Zielplattform (ARM Cortex-M4) eine andere Architektur als die Entwicklungsplattform (Linux-x86) aufweist, ist die Verwendung eines Cross-Compilers unerlässlich. Der Compiler muss Binärdateien erzeugen, die für den ARM-Befehlssatz optimiert sind.

```
1 CXX = arm-none-eabi-g++
2 OBJCOPY = arm-none-eabi-objcopy
3 SIZE = arm-none-eabi-size
4
5 CPU = -mcpu=cortex-m4 -mthumb -mfloat-abi=soft
6 CXXFLAGS = $(CPU) -I./include -Wall -Wextra -std=c++11 -g -O2 -fno-
  exceptions -fno-rtti
7 LDFLAGS = $(CPU) -nostdlib
8
9 SOURCES = src/main.cpp src/heartbeat.cpp src/mock_hal.cpp
10 OBJS = $(SOURCES:.cpp=.o)
11 ELF = heartbeat_sensor
12 HEX = heartbeat_sensor.hex
13 BIN = heartbeat_sensor.bin
14
15 all: $(ELF) $(HEX) $(BIN) size
16
17 $(ELF): $(OBJS)
18   $(CXX) $(LDFLAGS) -o $@ $^
19
20 $(HEX): $(ELF)
21   $(OBJCOPY) -O ihex $< $@
22
23 $(BIN): $(ELF)
```

```

24 $(OBJCOPY) -O binary $< $@
25
26 %.o: %.cpp
27 $(CXX) $(CXXFLAGS) -c $< -o $@
28
29 size: $(ELF)
30 @$ (SIZE) $<
31
32 clean:
33 rm -f $(OBS) $(ELF) $(HEX) $(BIN)
34
35 .PHONY: all clean size

```

Die wesentlichen Unterschiede zur Linux-Konfiguration sind:

- **Compiler-Tausch:** Anstelle des nativen g++ wird der arm-none-eabi-g++ eingesetzt. Dieser ist speziell dafür konzipiert, Code für ARM-Mikrocontroller ohne Betriebssystem zu erzeugen.
- **Architektur-Flags (CPU):** Durch Flags wie -mcpu=cortex-m4 und -mthumb wird der Compiler angewiesen, den spezifischen Befehlssatz des STM32-Kerns zu nutzen.
- **Spezielle Compiler-Optionen:** -fno-exceptions und -fno-rtti wurden hinzugefügt, da C++-Exceptions und RTTI (Run-Time Type Information) auf Mikrocontrollern aufgrund des limitierten Speichers und fehlender Laufzeitunterstützung meist deaktiviert werden.
- **Output-Formate:** Ein Linux-Programm ist normalerweise ein ausführbares ELF-File. Für den Mikrocontroller benötigen wir jedoch zusätzliche Formate (.hex oder .bin), die mittels arm-none-eabi-objcopy aus dem ELF-File extrahiert werden.

Im Gegensatz zum Standard-Linux-Build müssen diese Binärdateien in einem weiteren Schritt mit einem sogenannten Linkerscript (.ld-Datei) verbunden werden. Erst durch dieses Skript weiß der Mikrocontroller, an welche Speicheradresse (Flash oder RAM) die einzelnen Programmteile geschrieben werden müssen. Das Linkerscript wurde nicht in den Compile-Prozess integriert.

Zusammenfassend erlaubt dieses Makefile eine saubere Trennung zwischen der lokalen Logik-Validierung und der hardwarenahen Cross-Compilation, womit die Brücke vom PC zum eingebetteten System geschlagen wird.

5.3 Anpassungen in Autotools

Um eine Cross-Compilation mit Autotools zu gewährleisten müssen sowohl Anpassungen an der config.ac, als auch im Makefile vorgenommen werden. Autotools ist primär für die Portierung von Programmen zwischen unterschiedlichen Unix-Systemen geeignet. Für diesen Zweck erzeugt es regelmäßig Testfunktionen, um die Funktion des Compilers zu gewährleisten. Da der Cross-Compiler **arm-none-eabi** genutzt wurde und dieser Bare-Metal Code erzeugt, ist der Code für das Betriebssystem nicht lesbar und es tritt ein Fehler auf. Somit musste für die erfolgreiche Erzeugung eines Autotools-Makefile diese Art von Compiler Prüfung deaktiviert werden. Somit kann zwar das Makefile erzeugt werden, aber die Pipeline wird fehleranfälliger.

5.3.1 config.ac

```
1 AC_INIT([heartbeat_sensor], [1.0], [you@example.com])
2 AM_INIT_AUTOMAKE([-Wall -Werror foreign])
3
4 AC_NO_EXECUTABLES
5
6 AC_PROG_CC
7 AC_PROG_CXX
8
9 AC_CHECK_TOOL([OBJCOPY], [objcopy])
10 AC_CHECK_TOOL([SIZE], [size])
11
12 CPU_FLAGS="-mcpu=cortex-m4 -mthumb -mfloat-abi=soft"
13 AC_SUBST([CPU_FLAGS])
14
15 AC_CONFIG_FILES([Makefile src/Makefile])
16 AC_OUTPUT
```

- **AC_NO_EXECUTABLES:** Dieser Befehl verhindert, dass Autotools versucht kleine Testprogramme auszuführen. Somit können Fehlermeldungen verhindert werden und das Makefile wird generiert.
- **AC_CHECK_TOOL:** Dieser Befehl sucht nach dem arm-none-eabi-objcopy, wodurch das Erstellen von .bin und .hex Dateien ermöglicht wird.
- **CPU_Flags:** Hiermit wird die Architektur der CPU des Mikrocontrollers definiert und entsprechende Flags im Compile-Prozess.

5.3.2 Makefile.am

```
1 bin_PROGRAMS = heartbeat_sensor
2
3 heartbeat_sensor_SOURCES = main.cpp heartbeat.cpp mock_hal.cpp
4
5 AM_CXXFLAGS = $(CPU_FLAGS) -I$(top_srcdir)/include -Wall -Wextra -std=c++11 -g -O2 -fno-exceptions -fno-rtti
6 AM_LDFLAGS = $(CPU_FLAGS) -nostdlib --specs=nosys.specs
7
8 all-local: heartbeat_sensor.hex heartbeat_sensor.bin
9   @$ (SIZE) heartbeat_sensor$(EXEEXT)
10
11 heartbeat_sensor.hex: heartbeat_sensor$(EXEEXT)
12   $(OBJCOPY) -O ihex $< $@
13
14 heartbeat_sensor.bin: heartbeat_sensor$(EXEEXT)
15   $(OBJCOPY) -O binary $< $@
16
17 clean-local:
18   rm -f *.hex *.bin
19
```

- **AM_CXXFlags/ AM_LDFLAGS:** Diese Variablen fügen spezielle Parameter für den Mikrocontroller hinzu. Dabei sorgt nosys.specs dafür, dass keine Betriebssystem-Bibliothek eingebunden wird, da der Mikrocontroller Bare Metal betrieben wird.
- **all-local:** Hier findet die Verknüpfung mit dem OBJCOPY in der config.ac statt, wodurch die .hex und .bin Dateien erzeugt werden. In Zeile 11 und 14 werden die kompilierten ELF-Dateien in binär und hex Dateien umgewandelt.
- **„Size:** Erzeugt eine Ausgabe der Dateigröße im Terminal, wenn der Build abgeschlossen ist.

Die STM Autotools kann wie folgt ausgeführt werden:

```
1 autoreconf -i
2 ./configure --host=arm-none-eabi
3 make
```

Im Anschluss werden im src Ordner die heartbeat_sensor.hex und .bin erzeugt.

6 Auswertung

Die beiden zu programmierenden Varianten *manual makefile* und *Autotools* konnten erfolgreich umgesetzt werden. Die korrekte Funktionsweise wurde, soweit wie möglich, getestet. Alle Tests erfolgten dabei nur auf dem programmierenden Gerät, nicht auf der späteren Hardware. Die eingesetzte HAL-Bibliothek wurde dementsprechend nur durch die selbst geschriebene *mock-hal.cpp* simuliert.

Im Aufbau und der Funktionsweise ergeben sich einige Unterschiede. Das *manual makefile* ist deutlich kleiner und einfacher aufgebaut. Der zeitliche Aufwand für das manuelle Makefile und die *Autotools*-Pipeline ist für dieses kleine Projekt nahezu identisch. Für größere Anwendungsfälle wird die Zeitersparnis durch die *Autotools*-Variante allerdings erheblich zunehmen, da weniger händische Zuweisungen erstellt werden müssen. Auch die Übersichtlichkeit ist bei *Autotools* bedeutend besser als die des manuellen Makefiles. Neben diesen beiden Vorteilen implementiert *Autotools* mehrere Überprüfungen, die am Ende zu einem stabileren Buildprozess führen. Derartige Prüfungen händisch in ein Makefile zu integrieren, erfordert ein tiefes Systemverständnis.

Für kleine Projekte eignen sich daher auch beide Varianten. Für größere Projekte eignet sich eher die Variante über *Autotools*, da dabei Zeit gespart wird. Die Systemressourcen sind nicht ganz so relevant, da für größere Projekte tendenziell ohnehin mehr Ressourcen benötigt werden. *Autotools* bietet außerdem eine robustere Erstellung des Makefiles mit mehr Überprüfungen, als händisch realisierbar wäre.

Crosscompiling wird dann sinnvoll, wenn auf unterschiedlichen Hard- und Softwaredistributionen für eine dritte Hardware programmiert wird. Dies ist vor allem bei Projekten mit mehreren Programmierern der Fall. Des Weiteren können so gleichzeitig verschiedene Teilfunktionen programmiert werden, auch wenn die Zielhardware nur einmal zur Verfügung stehen sollte.